UNIVERSIDAD SERGIO ARBOLEDA

PARCIAL 2

LENGUAJES DE PROGRAMACION

DOCENTE

JOAQUÍN FERNANDO SÁNCHEZ CIFUENTES

JULIAN CAMILO CARO HERRERA

PRIMER PUNTO

1. **Definición de Símbolos Terminales**

Palabras clave

CREATE, READ, UPDATE, DELETE, WITH, SET, WHERE,
AND, OR, NOT, IN,
true, false, null

Operadores

Comparación: ==, !=, >, <, >=, <=, IN
Asignación: =
Acceso a campos: .
Lógicos: AND, OR, NOT

Delimitadores

{ } (objetos)
[ ] (listas)
( ) (agrupación)
, (separador)
: (asociación clave-valor)
; (terminador de sentencia)

Literales

Cadenas: "texto"
Números: enteros y reales (123, 45.67)
Booleanos: true, false
Nulo: null
1.5 Identificadores

Secuencias alfanuméricas que comienzan con una letra:
identifier ::= letter (letter | digit | "_")*

2. Análisis Técnico del Diseño

2.1 Eliminación de Recursividad por la Izquierda

se eliminó toda forma de recursividad por la izquierda, la cual provoca ciclos infinitos

Por ejemplo, una problemática sería:

expr → expr OR term

Esta estructura fue transformada en una forma iterativa equivalente:

or_expr ::= and_expr ("OR" and_expr)*

Asociatividad izquierda
Evita recursión por la izquierda
Es directamente procesable mediante parsers predictivos

El mismo enfoque se aplica en múltiples reglas como:
program ::= statement (";" statement)* ";"
assignments ::= assignment ("," assignment)*

Resolución de Ambigüedad y Precedencia de Operadores

La gramática implementa un diseño jerárquico de expresiones.la precedencia de operadores sin necesidad de estructuras externas como tablas de precedencia.
La jerarquía definida es:

expression → or_expr
or_expr → and_expr ("OR" and_expr)*
and_expr → not_expr ("AND" not_expr)*
not_expr → ["NOT"] comparison_expr

Este diseño garantiza:

OR---- tiene la menor precedencia
AND ----tiene precedencia intermedia
NOT---- tiene la mayor precedencia lógica
Las comparaciones (==, >, IN, etc.) se evalúan a nivel inferior:

comparison_expr ::= primary_expr (comp_operator primary_expr)?

Recursividad

El lenguaje permite estructuras de datos anidadas mediante recursividad indirecta en la definición de value:

```
value ::= literal
        | json_object
        | json_array

json_object ::= "{" [pair ("," pair)*] "}"
pair        ::= identifier ":" value

json_array  ::= "[" [value ("," value)*] "]"
```

Esta definición permite:

Objetos dentro de objetos
Listas dentro de objetos
Listas de listas

Ejemplo válido:

```
{ usuario: { nombre: "Ana", hobbies: ["leer", "programar"] } }
```

Ejemplo de Derivación y AST

Consulta de ejemplo

```
UPDATE usuarios
SET direccion.ciudad = "Bogotá", edad = 30
WHERE usuario.edad >= 18 AND NOT usuario.activo == false;
```

Estructura del AST

```
UPDATE
├── colección: usuarios
├── SET
│   ├── asignación
│   │   ├── campo: direccion.ciudad
│   │   └── valor: "Bogotá"
│   └── asignación
│       ├── campo: edad
│       └── valor: 30
└── WHERE
    └── OR/AND árbol lógico
        └── AND
            ├── comparación (>=)
            │   ├── usuario.edad
            │   └── 18
            └── NOT
                └── comparación (==)
                    ├── usuario.activo
                    └── false
```

SEGUNDO PUNTO

Implementación y Pruebas de la Gramática y visualización

Casos de Prueba

```
 GNU nano 7.2                                                    entrada.txt
CREATE usuarios WITH { nombre: "Juan", edad: 25 };

UPDATE usuarios SET direccion.ciudad = "Bogotá", edad = 30
WHERE usuario.edad >= 18 AND NOT usuario.activo == false;

READ usuarios WHERE edad > 20;

DELETE usuarios WHERE edad < 18;
```

Figura 1 (Entrada txt)

Prueba 1: Creación de documento

CREATE usuarios WITH { nombre: "Juan", edad: 25 };

Resultado:

- Se reconoce correctamente la instrucción CREATE
- El objeto JSON es parseado como estructura jerárquica

Prueba 2: Actualización con condición compleja

UPDATE usuarios
SET direccion.ciudad = "Bogotá", edad = 30
WHERE usuario.edad >= 18 AND NOT usuario.activo == false;

Validaciones:

- Acceso a campos anidados (direccion.ciudad)
- Varias asignaciones
- Uso de operadores lógicos (AND, NOT)
- Correcta precedencia de operadores

Prueba 3: Consulta con filtro

READ usuarios WHERE edad > 20;

Prueba 4: Eliminación con condición

DELETE usuarios WHERE edad < 18;

Resultado:

- La instrucción DELETE es válida
- El filtro es correctamente procesado

Visualización del Árbol Sintáctico

Para facilitar la interpretación de los resultados, el árbol de análisis sintáctico se imprimió en formato jerárquico indentado.

```
;
statement
  delete_stmt
    DELETE
    identifier
      usuarios
    where_clause
      WHERE
      expression
        or_expr
          and_expr
            not_expr
              comparison_expr
                primary_expr
                  field_access
                    identifier
                      edad
                comp_operator
                  <
                primary_expr
                  literal
                    18
;
```

Figura 2 (Visualización Árbol)

Este formato permite observar claramente la estructura interna de cada instrucción y la relación entre sus componentes.

Punto 3: Ambigüedad en la Gramática de *if-then-else*

Análisis de Ambigüedad

Se parte de la siguiente gramática:

```
prop -> if expr then prop
     | prop_emparejada

prop_emparejada -> if expr then prop_emparejada else prop
                 | otras
```

Esta gramática intenta resolver el problema del *dangling else*, sin embargo, sigue siendo ambigua**.**

Cadena de prueba

Se analiza la cadena:

```
if E1 then if E2 then S1 else S2
```

Asociación al *if* interno

```
if E1 then (if E2 then S1 else S2)
```

- El else se asocia con el if más cercano (if E2)

Derivación conceptual:

- `prop → if expr then prop`
- `prop → prop_emparejada`
- `prop_emparejada → if expr then prop_emparejada else prop`

Asociación al *if* externo

```
(if E1 then if E2 then S1) else S2
```

En esta interpretación:

- El else se asocia con el if externo (if E1)
- Se obtiene una estructura sintáctica diferente

Derivación:

- `prop → prop_emparejada`
- `prop_emparejada → if expr then prop_emparejada else prop`

Conclusión

Dado que la misma cadena admite dos árboles de derivación distintos, La gramática es ambigua, ya que no define de manera única la asociación del else.

Solución

Una restricción estructural que garantice que:

Cada else se asocie con el if más cercano que no tenga pareja

Esto elimina cualquier ambigüedad.

Implementación de la Solución

La gramática corregida es la siguiente:

```
prop -> matched
     | unmatched

matched -> if expr then matched else matched
         | otras

unmatched -> if expr then prop
           | if expr then matched else unmatched
```

Justificación

Eliminación del *dangling else*

La clave está en la regla:

```
unmatched -> if expr then matched else unmatched
```

Esta producción garantiza que:

- Un else solo puede aparecer cuando se está resolviendo un if pendiente
- La asociación siempre se realiza con el if más cercano no emparejado

Eliminación de ambigüedad

La gramática ahora:

- Restringe las derivaciones posibles
- Evita interpretaciones múltiples
- Garantiza una única estructura sintáctica por cadena

Analizadores

La nueva gramática es adecuada porque:

- Elimina conflictos de tipo *shift/reduce*
- Define explícitamente la estructura de las proposiciones
- Permite decisiones de parsing sin ambigüedad

Punto 4: Análisis y Comparación de Métodos de Parsing

Gramática de Calculadora

Definición en BNF (con precedencia)

```
<expr>   ::= <expr> "+" <term>
           | <expr> "-" <term>
           | <term>

<term>   ::= <term> "*" <factor>
           | <term> "/" <factor>
           | <factor>

<factor> ::= "(" <expr> ")"
           | <num>

<num>    ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Esta gramática define correcta precedencia de operadores:

- Multiplicación y división tienen mayor precedencia que suma y resta
- El uso de paréntesis permite modificar la precedencia

Transformación a Forma Normal de Chomsky (FNC)

```
S  -> E

E  -> E X1 | E X2 | T
X1 -> PLUS T
X2 -> MINUS T

T  -> T X3 | T X4 | F
X3 -> MUL F
X4 -> DIV F

F  -> LPAREN X5 | NUM
X5 -> E RPAREN

PLUS   -> "+"
MINUS  -> "-"
MUL    -> "*"
```

```
DIV    -> "/"
LPAREN -> "("
RPAREN -> ")"

NUM -> DIGIT
DIGIT -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Esta forma cumple con la estructura requerida por el algoritmo CYK:

- Producciones del tipo A → BC
- Producciones del tipo A → a

Análisis Comparativo: CYK vs Predictivo LL(1)

Criterio	CYK	Predictivo LL(1)
Estructura de datos	Matriz triangular `n × n` con conjuntos de variables	Pila explícita + tabla de parsing `M[A,a]`
Mecánica de decisión	Evaluación exhaustiva de subcadenas y particiones	Selección determinista usando símbolo *lookahead*
Tipo de algoritmo	Dinámico	Determinista
Complejidad temporal	$O(n^3 \cdot \lvert G \rvert)$	$O(n)$
Uso de memoria	Alto (debido a la matriz completa)	Bajo (estructura lineal)
Generalidad	Soporta cualquier gramática en FNC	Solo gramáticas LL(1)

```
julian@julian-VirtualBox:~/LENGUAJES DE PROGRAMACION/2Parcial/4punto$ python3 parser.py
=== INICIO DEL ANÁLISIS ===

Match: if
Match: ID
Match: then
Match: ID
Match: :=
Match: NUM
Match: else
Match: ID
Match: :=
Match: NUM

 Cadena aceptada.
```

QUINTO PUNTO

Implementación del algoritmo parea

El algoritmo parea (match) es el encargado de validar que el token actual de la entrada coincide con el símbolo terminal esperado. En caso de coincidencia, el analizador avanza al siguiente token de lo contrario, se genera un error sintáctico.

Gramática

proposicion -> asignacion | condicional
asignacion -> ID ':=' expresion
condicional -> 'if' expresion 'then' proposicion ('else' proposicion)?
expresion -> ID | NUM

```
julian@julian-VirtualBox:~/LENGUAJES DE PROGRAMACION/2Parcial/5Punto$ python3 parea.py
 proposicion condicional
 condicional
 Match: if
 expresion
 Match: ID
 Match: then
 proposicion condicional
 condicional
 Match: if
 expresion
 Match: ID
 Match: then
 proposicion asignacion
 asignacion
 Match: ID
 Match: :=
 expresion
 Match: NUM
 Match: else
 proposicion asignacion
 asignacion
 Match: ID
 Match: :=
 expresion
 Match: NUM

 Cadena aceptada completamente.
```